

Final Report — Chapter 11: Hardware/Software Co-Design

CECS 460 — System on Chip | Spring 2026 Team: Nathan Sarkozy and Christian Vanegas **Chapter 11: Hardware/Software Co-Design and Design Flow**

1. Summary

This module teaches hardware/software co-design by having students experience a real partitioning failure and fix it themselves. Students wire a potentiometer and LCD to an ESP32, observe the display lagging under CPU load (Step 2), measure the sample rate degradation via Serial Monitor (Step 3), then reflash with DMA-based ADC firmware and watch the lag disappear under the same load (Step 4). Step 5 asks them to generalize from their own measured data into a decision rule.

The module replaces the original Chapter 11 lab, which demonstrated the same concept through an AES-128 software vs. hardware benchmark. The new approach makes the consequence of poor partitioning physically visible and interactive, requiring no oscilloscope or specialized equipment beyond a standard breadboard kit and a 16×2 LCD.

2. Design Decisions

2.1 Why the "overwhelm → rescue" arc

The original Ch11 lab asked students to run firmware, wait for MQTT telemetry, and compare two numbers (sw_us vs. hw_us). This is accurate but passive — the student is a spectator. The "overwhelm → rescue" arc makes the student a participant: they build the circuit, they feel the failure, they flash the fix. The core co-design insight (dedicated hardware frees the CPU) lands harder when students have just watched a display lag and then stop lagging with zero wiring changes.

2.2 Why potentiometer + LCD instead of AES benchmark

AES encryption is correct as a co-design example but hard to visualize. A lagging LCD display communicates the problem instantly to anyone watching. The potentiometer also makes the demo repeatable and interactive — any visitor at the expo can turn the knob and see the lag themselves, without needing context.

2.3 Why DMA instead of a hardware AES engine

DMA is the most universally applicable hardware offload concept on any microcontroller. AES acceleration is ESP32-specific and only useful for one task. DMA teaches the general principle: any data-movement task that would otherwise occupy the CPU can be handed to a DMA controller. The student who understands DMA can apply the concept to SPI, I2S, UART, and ADC contexts — not just AES.

2.4 Why Serial Monitor instead of oscilloscope

An oscilloscope is the right tool to visualize timing jitter, but not every classroom has them and they require setup time. Serial Monitor is available on every machine that can run Arduino IDE. The `step1_baseline` firmware was modified to print measured period and frequency after every blink cycle so students get a numeric measurement without scope hardware. Steps 2–4 already used Serial Monitor for sample rate reporting, making the tool choice consistent across all steps.

2.5 Why start with a plain LED blink

Step 1 is pedagogically important. Students commit to an answer (why isn't it exactly 1 Hz?) before understanding the full concept. That investment makes Steps 2–4 meaningful — they're not just following instructions, they're testing a hypothesis they already formed. Starting with a visually simpler step also gives slower teams a confidence win before the more complex wiring of Steps 2–4.

3. Hardware Choices

Component	Choice	Rationale
ADC input	Potentiometer on GPIO 34	Analog input that any user can vary manually; makes the lag interactive
Display	16×2 parallel LCD (TC1602A, HD44780-compatible) driven in 4-bit mode	Available in most teaching kits; uses the standard <code>LiquidCrystal</code> library built into Arduino IDE (no third-party install for students); shows a visible number that lags
DMA method	ESP32 I2S peripheral in ADC-DMA mode	Built into ESP32; no external hardware required; well-documented in <code>arduino-esp32 3.x</code>
CPU load	Floating-point loop (5,000 iterations)	Produces reliable, adjustable load; easy to understand; compiler-resistant with volatile result

Full BOM with costs and sourcing: [hardware/BOM.md](#)

4. System Files Created or Modified

File	Status	Description
lesson_package/lesson.json	Created	Full 5-step lesson with interactive moments and short-answer prompts
lesson_package/grading.json	Created	Keyword-weighted rubric for all 5 questions with exemplars
hardware/starter_code/step1_baseline/step1_baseline.ino	Modified	Added Serial Monitor period/frequency output; removed oscilloscope dependency
hardware/starter_code/step2_overload/step2_overload.ino	Created	Potentiometer + LCD + CPU load demo; prints sample rate to Serial
hardware/starter_code/step4_accelerator/step4_accelerator.ino	Created	Same hardware + load as Step 2, ADC via I2S DMA; prints DMA sample rate
hardware/starter_code/README.md	Created	Build/flash guide with wiring tables and common issues
lab/step1_handout.md through step5_handout.md	Created	Student-facing lab sheets for all 5 steps
lesson_package/instructor_notes/step1.md	Exists	Instructor notes for Step 1

File	Status	Description
docs/instructor_guide.md	Created	Full deployment guide for next semester's instructor
hardware/BOM.md	Exists	Complete bill of materials with sourcing and per-team cost
tests/test_grading.py	Modified	Expanded to test all 5 grading rubric questions

5. Testing Process and Evidence

5.1 Grading rubric validation

The `tests/test_grading.py` script validates each question's keyword rubric against full-credit, partial-credit, and disqualifying sample answers. Run with:

```
python3 tests/test_grading.py
```

Expected output: all test cases pass. Results are saved to `testing_evidence/`.

5.2 Hardware verification

The end-to-end hardware run is documented in `testing_evidence/`. Each step has a Serial Monitor screenshot confirming expected output:

- **Step 1:** `step1_baseline.ino` flashed; Serial Monitor reports 0.996–1.004 Hz over 10 cycles. Confirmed timing drift is software-side, not hardware.
- **Step 2:** `step2_overload.ino` flashed; LCD visibly lags the potentiometer; Serial Monitor shows [SW] Sample rate: ~40–80 Hz under the floating-point load.
- **Step 4:** `step4_accelerator.ino` flashed with `USE_DMA=1`; same wiring, same load, but Serial Monitor reports [DMA] Sample rate: ~400–500 Hz and the LCD tracks the knob without visible lag.

5.3 End-to-end server pass

The Step 4 firmware connects to the classroom WiFi (CECS) once the DMA sample rate sustains above 200 Hz for 5 seconds, then publishes `{answers: {q4_lab_pass: "PASS"}}` to

c460_ch11_codesign/{slot}/answer . The classroom server's `scoring_engine.py` (patched to handle both grading schemas) scores it 10/10 against `ch11Lab/grading.json` , and the instructor dashboard at `/cecs460/instructor` displays the slot as passing.

A screenshot of the dashboard recording the pass is in `testing_evidence/step4_dashboard_pass.png` . The full pass flow — firmware flash → DMA threshold met → MQTT publish → server scoring → dashboard update — runs without any instructor intervention.

5.4 Failure modes encountered during testing

- **Schema mismatch crash.** The classroom server's `scoring_engine.py` originally only recognized the legacy grading schema (`q["id"]` , `q["points"]` , `q["keywords"]`); the new schema (`q["question_id"]` , `q["max_points"]` , `q["scoring"]["required_concepts"]`) caused a `KeyError` on every MQTT submission. Fixed by changing field accesses to `q.get("id")` or `q.get("question_id", "")` so both schemas coexist.
- **DHCP IP drift.** The Mango router's DHCP lease for the laptop changed between testing sessions, breaking the firmware's `MQTT_HOST` until reflashed. Documented in the firmware README; long-term fix is to switch to a static lease or mDNS resolution.

6. Recommendations for Future Improvement

1. **Publish richer telemetry beyond pass/fail.** The Step 4 firmware currently sends only a `PASS` token. Publishing the actual sample-rate-before / sample-rate-after pair would let the dashboard plot a class-wide histogram of speedups, and would let the rubric grade the student's *measured improvement ratio* rather than just the threshold crossing.
 2. **Add a WS2812 LED bar graph.** A 16-pixel LED bar graph is more dramatically visual than an LCD number. Step 2's failure would be immediately obvious to anyone across the room, making it more effective as an expo demo.
 3. **Add a configurable load level.** A serial command to change `LOAD_STRENGTH` at runtime would let students find the exact threshold where sample rate drops below a target — a more investigative version of Steps 2–3.
 4. **Record canonical Serial Monitor captures.** Reference screenshots of expected Serial output for each step give instructors a quick sanity check and give students something to compare against when their output looks unexpected.
 5. **Add a dual-core comparison step.** Pinning the load task to Core 0 and the ADC poll to Core 1 (FreeRTOS) is an alternative partitioning option that doesn't require DMA. A "Step 4b" comparing the two approaches would deepen the partition discussion in Step 5.
-

7. Acknowledgments

AI tools (Claude) were used to assist with drafting lesson content, grading rubric structure, and code review. All pedagogical decisions, hardware choices, wiring design, and final content were made and verified by the team. All hardware testing was performed by the team on physical ESP32 hardware. Per-area ownership is documented in `docs/contribution_statement.md`.